# F26-186-TraceLearn

FYP-I Mid Report: Iteration 1


Project Team

Rayaan Raza i23-0535
Umaima Zeeshan i23-0790
Eman Ihsan i23-0530


Session 2026-2027

Supervised by

Dr. Ali Zeeshan Ijaz





**Department of Computer Science**

**National University of Computer and Emerging Sciences**
**Islamabad, Pakistan**


**October, 2026**

List of Figures ..... 4
List of Tables ..... 5
Introduction ..... 6
1.1 Existing Solutions ..... 6
1.2 Problem Statement ..... 7
1.3 Scope ..... 7
1.4 Modules ..... 8
1.4.1 Module 1: Authentication and Role-Based Access ..... 8
1.4.2 Module 2: Practice Interface ..... 8
1.4.3 Module 3: Sandboxed C++ Execution and Test Runner ..... 8
1.4.4 Module 4: Evidence Extraction ..... 9
1.4.5 Module 5: Diagnosis and Learner State ..... 9
1.4.6 Module 6: Hints and Teacher Review ..... 9
1.4.7 Module 7: Classrooms, Data and CI/CD ..... 9
1.5 Work Division ..... 9
1.5.1 Iteration Plan and Progress ..... 10
1.6 Proposal Panel Actions ..... 12
Project Requirements ..... 13
2.1 Use Cases ..... 13
2.2 Functional Requirements ..... 15
2.2.1 Module 1: Authentication and Role-Based Access ..... 15
2.2.2 Module 2 and 3: Practice Interface and Execution ..... 15
2.2.3 Module 4 and 5: Evidence, Diagnosis and Learner State ..... 16
2.2.4 Module 6 and 7: Hints, Review, Classrooms and CI ..... 16
2.3 Non-Functional Requirements ..... 16
2.3.1 Security ..... 16
2.3.2 Performance ..... 17
2.4 Acceptance Criteria and Traceability ..... 17
System Overview ..... 19
3.1 Architectural Design ..... 19
3.1.1 User Layer ..... 19
3.1.2 Application Services ..... 19
3.1.3 Diagnostic Intelligence ..... 20
3.1.4 Component Interfaces ..... 20
3.1.5 Design Decisions ..... 20
3.2 Data Design ..... 21

3.3 Process Design ..... 23
3.4 Behaviour Design ..... 24
3.5 Design Evolution ..... 24
Implementation and Testing ..... 25
4.1 Setup and Execution ..... 25
4.2 Main Files and Functions ..... 25
4.3 Algorithm Design ..... 26
4.3.1 Sandboxed C++ Compilation and Test Execution ..... 26
4.3.2 Submission Analysis Pipeline ..... 27
4.3.3 Evidence Fusion and Diagnosis ..... 27
4.3.4 Learner-State Update ..... 27
4.3.5 Hint Selection ..... 28
4.4 Technical Decisions and Limitations ..... 28
4.5 External APIs, Tools and Dependencies ..... 28
4.6 Testing Details ..... 29
4.6.1 Unit and Contract Testing ..... 29
4.6.2 Integration Testing ..... 30
4.6.3 Live Demonstration Plan ..... 31
4.6.4 Failure Case Discussion ..... 31
4.7 Risk, Security and Privacy ..... 31
4.8 GenAI Use Disclosure ..... 32
4.9 Research and Citation Verification ..... 32
Conclusions and Future Work ..... 34
5.1 Conclusion ..... 34
5.2 Future Work ..... 34
References ..... 35

# List of Figures

Figure 2.1: Use-Case Diagram ..... 15
Figure 3.1: System Architecture of TraceLearn (v2, October 2026) ..... 19
Figure 3.2: Entity-Relationship Diagram of TraceLearn (as built) ..... 21
Figure 3.3: Database Schema of TraceLearn ..... 22
Figure 3.4: Activity Diagram of One Submission ..... 23
Figure 3.5: State-Transition Diagram of a Learner Misconception State ..... 24

# List of Tables

Table 1.1: Comparison of Existing Solutions ..... 6
Table 1.2: Work Breakdown Planned at the Proposal ..... 10
Table 1.3: Work Completed in Iteration 1 (from the Commit History) ..... 10
Table 1.4: Iteration Plan from the Proposal ..... 10
Table 1.5: Iteration 1 Progress ..... 11
Table 1.6: Blockers in Iteration 1 ..... 11
Table 1.7: Proposal Action Register ..... 12
Table 2.1: Use Cases for TraceLearn (Iteration 1) ..... 13
Table 2.2: Security Requirements ..... 16
Table 2.3: Performance Requirements ..... 17
Table 2.4: Requirements and Acceptance Evidence ..... 17
Table 2.5: Requirement Traceability ..... 18
Table 3.1: Main API Endpoints ..... 20
Table 3.2: Architecture and Interface Decisions ..... 20
Table 3.3: Key Entities and Their Relationships ..... 22
Table 3.4: Design Artefacts and Changes since the Proposal ..... 24
Table 4.1: Main Files of the Core Module ..... 25
Table 4.2: Summary of Algorithms Implemented in Iteration 1 ..... 26
Table 4.3: External Tools and Dependencies ..... 28
Table 4.4: Backend Test Suites ..... 29
Table 4.5: Selected Test Cases ..... 29
Table 4.6: Testing Statistics Overview ..... 30
Table 4.7: Live Demonstration Steps ..... 31
Table 4.8: GenAI Artefact Register ..... 32

# Chapter 1

# Introduction

This report presents Iteration 1 of TraceLearn for the FYP-I mid evaluation. TraceLearn is a programming practice platform. Students write C++ code and submit it. The system runs the code, examines the result and finds possible misconceptions. It uses the history of the student, not only the current submission. The research question is: does the history of a student give a better diagnosis than the current submission alone?

Iteration 1 builds one vertical slice of the system. An authenticated student opens an assigned C++ problem in the Monaco editor and submits code. The server compiles and runs the code in a sandbox and runs the visible, hidden and diagnostic tests. The system then extracts evidence, makes two diagnoses (current-only and history-aware), updates the learner state and selects a hint. A teacher can examine the diagnosis and record a review. Continuous integration (CI) runs the test suite on each change.

The evaluation version is the `main` branch of https://github.com/Rayaan-Raza/TraceLearn at commit fb94119.

## 1.1 Existing Solutions

The team studied research on misconception detection, automated grading and datasets of student code. Table 1.1 gives the systems and datasets that the team examined. The team checked each fact in the table against the original source (see Section 4.9).

Table 1.1: Comparison of Existing Solutions

| System or dataset | What it does | Limitation for TraceLearn |
|---|---|---|
| McMining [1] | Two LLM tools (McMiner-S, McMiner-M) find a misconception from a bag of 4 to 8 code samples of one student. The benchmark has 67 misconceptions and 1,675 code samples (1,063 with a misconception). | Python only. The samples were made by injecting misconceptions into correct code, not by students. No learner state over time. |
| Pass/fail autograders and online judges (verdicts as in Project CodeNet [3]) | Run the code against tests and give a verdict such as Accepted, Wrong Answer or Time Limit Exceeded. | A verdict does not say why the code failed. Each submission is judged alone. |
| Knowledge tracing on programming data (CSEDM 2019 / ITAP, as listed in the worksheet) | Predicts whether a student will solve the next problem from the attempt history. | Models skill, not a specific false belief. Recheck this row against a cited paper before submission. |
| Project CodeNet [3] | 13,916,868 submissions on 4,053 problems; 8,008,527 in C++. | Judge verdicts only; no misconception labels. |

| System or dataset | What it does | Limitation for TraceLearn |
|---|---|---|
| PROGpedia [4] | 9,117 submissions to 16 exercises in Java, C, C++ and Python, labelled Accepted, Wrong Answer or Runtime Error. | Verdicts, not misconceptions. No attempt order for each student. |
| CodeStream [5] | 5,482 submissions from 202 students on 46 problems (C 2,112, Java 2,086, C++ 1,284) with attempt number and verdict sequence. | No misconception labels. C++ is the smallest part. |

These systems and datasets give verdicts, single-submission feedback or synthetic Python samples. None of them keeps a misconception state for each student across problems, and none of them combines execution, test and static evidence for C++. This is the gap that TraceLearn addresses.

## 1.2 Problem Statement

**Students repeat the same misconception across problems, but platforms judge each submission on its own.** Beginner students often give similar incorrect outputs for different reasons. The same false belief also appears in different code over time. A pass/fail grade cannot find this difference, and a teacher cannot see which student has which misconception and whether it continues.

The complex computing challenge is to find and update a hidden misconception state from evidence that is incomplete, mixed and sometimes in conflict. The system must not use future information, and it must explain its result. One submission or one simple rule is not sufficient.

## 1.3 Scope

The scope of Iteration 1 is authenticated C++ practice with these parts:

- Role-based access for students and teachers (JWT session in an httpOnly cookie with a CSRF token).
- The Monaco editor in C++ mode.
- Server-side compilation and execution of C++17 in a bubblewrap sandbox with CPU, memory, output and wall-clock limits.
- Visible, hidden and diagnostic tests for each problem. Students never receive the input or output of hidden and diagnostic tests.
- Evidence extraction from five sources (test, static, execution, sequence and history).
- Two diagnoses for each submission: current-only and history-aware.
- A learner state for each student and misconception, with a history row for each change.

- Template hints at three levels and a teacher review that does not change the original diagnosis.
- Classrooms and assignments, and a CI/CD pipeline.

These items are excluded from Iteration 1: an XGBoost classifier, any LLM in the diagnosis, support for more than one language in the practice pages, plagiarism detection, leaderboards and a mobile application. The report makes no claim that the system helps real students; all evaluation data is synthetic.

The success criteria for Iteration 1 are:

1. All automated tests pass in CI. Result: 191 of 191 backend tests pass on Linux with bubblewrap; CI run 38061789808 passed on commit fb94119.
2. A live C++ submission gives stored evidence and a diagnosis. Result: shown in the live demonstration (Section 4.6.3).
3. The system processes a timeout or a compile error safely. Result: an infinite loop gives `timeout`, code that does not compile gives `syntax_error`, and the API continues to operate.
4. Each reference solution of the 40-problem C++ bank is diagnosed as NO_MISCONCEPTION. Result: 40 of 40.

## 1.4 Modules

The system is divided into seven modules. Each module has one main function.

### 1.4.1 Module 1: Authentication and Role-Based Access

1. Registration and sign-in. A teacher account needs an invite code (TEACHER_SIGNUP_CODE).
2. A JWT with the role, kept in an httpOnly cookie; a CSRF token on each request that changes data.
3. Role checks on each endpoint (401 without a session, 403 for the wrong role, 404 for an item that the user cannot see). Sign-out cancels all tokens of the user.

### 1.4.2 Module 2: Practice Interface

1. The student pages: dashboard, practice list, problem page with the Monaco editor in C++ mode, result page, submissions and progress.
2. The teacher pages: class overview, classroom, assign problem, students, misconceptions, submission detail and research.
3. The practice and assignment lists show C++ problems only.

### 1.4.3 Module 3: Sandboxed C++ Execution and Test Runner

1. Source checks: maximum 64 KB, and only standard headers in #include.

2. Compilation with `g++ -std=c++17 -O2 -Wall` and execution in bubblewrap (no network, no host files, empty environment).
3. Each test runs with its own wall-clock limit (2,000 ms for the bank problems) and a 256 MB memory limit. The output is compared token by token.
4. The status of each submission is one of `success`, `syntax_error`, `runtime_error`, `wrong_answer`, `timeout` or `system_error`.

### 1.4.4 Module 4: Evidence Extraction

1. Test evidence: failed visible, hidden and diagnostic tests; diagnostic tags map to a misconception.
2. Static evidence: Python `ast` patterns, and regular expressions plus g++ warnings for C++ (in a separate process with a 3-second limit).
3. Execution, sequence (earlier attempts on the same problem) and history (other problems) evidence.

### 1.4.5 Module 5: Diagnosis and Learner State

1. Weighted evidence fusion with fixed weights from `config/evidence_weights.yaml` (version poc-v1).
2. Two separate engines: current-only and history-aware. The history-aware engine reads only data from before the submission.
3. A learner state for each student and misconception (NEW, PERSISTENT, IMPROVING, RESOLVED, RECURRED, REQUIRES_TEACHER_REVIEW) and an append-only history with the reason for each change.

### 1.4.6 Module 6: Hints and Teacher Review

1. Three hint levels (guiding question, concept reminder, where to look). The first hint is level 1. A problem-specific hint replaces the template hint.
2. A teacher can confirm, reject, correct or mark a diagnosis uncertain. The original diagnosis does not change.

### 1.4.7 Module 7: Classrooms, Data and CI/CD

1. Classrooms with a join code, assignments for all or some students, and the C++ diagnostic bank of 40 problems (305 test cases, 32 misconception variants, 120 hints).
2. GitHub Actions: backend tests on Python 3.11 and 3.12 with g++ and bubblewrap, and frontend lint and build.

## 1.5 Work Division

Table 1.2 gives the division of work that the team planned at the proposal stage. Table 1.3 gives the work that each member completed in Iteration 1, taken from the commit history.

Table 1.2: Work Breakdown Planned at the Proposal

| Member | Planned tasks |
|---|---|
| Rayaan Raza | Initial POC (evidence extraction, diagnosis, learner model, hints); dataset audit and normalization; problem bank models, API and loader; per-test execution limits; the 40-problem C++ diagnostic bank and its verifiers; C++ scoring; deployment configuration |
| Eman Ihsan | Bubblewrap sandbox; hardened authentication and sessions; auth on every endpoint and the classroom API; security test suite; submission logging with basic evidence capture; frontend connected to the backend; CI/CD |
| Umaima Zeeshan | First RBAC (JWT, student and teacher roles); Monaco editor in C++ mode; server-side C++ compiler and run pipeline; Iteration 1 QA and fixes; QA evidence |

Table 1.3: Work Completed in Iteration 1

| Member | Completed work | Main commits |
|---|---|---|
| Rayaan Raza | Initial POC (evidence extraction, diagnosis, learner model, hints); dataset audit and normalization; problem bank models, API and loader; per-test execution limits; the 40-problem C++ diagnostic bank and its verifiers; C++ scoring; deployment configuration | bc22b0a, 1834765, 419e689, 0fcaa27, 8f0ec5e, e76e6bc, 12c3c26, 50077bc, 23b9825 |
| Umaima Zeeshan | Bubblewrap sandbox; hardened authentication and sessions; auth on every endpoint and the classroom API; security test suite; submission logging with basic evidence capture; frontend connected to the backend; CI/CD | c30e5aa, fc16065, af1ce9b, fe6557a, b88a962, 4102ed2, 179f325, fd15a4b, c983924 |
| Eman Ihsan | First RBAC (JWT, student and teacher roles); Monaco editor in C++ mode; server-side C++ compiler and run pipeline; Iteration 1 QA and fixes; QA evidence | be7897b, 5619739, 8ef0efb, fb94119, 016b4c2 |

The planned and the completed work are the same due to clear communication of expectations from day one.

### 1.5.1 Iteration Plan and Progress

Table 1.4 gives the four iterations from the proposal.

Table 1.4: Iteration Plan from the Proposal

| Iteration | Planned work |
|---|---|
| Iteration 1 | RBAC; Monaco integration; initial server-side C++ execution pipeline; schema normalization; draft misconception taxonomy; early-stage evidence extraction pipeline; submission logging with basic evidence capture |
| Iteration 2 | Problem and test bank expansion; annotation handbook and labeling protocol; learner-state modeling; misconception classifier (XGBoost); adaptive hint framework (initial levels); sandboxed C++ environment; submission view |
| Iteration 3 | Problem bank extension (toward 40 to 60 problems); classifier tuning; adaptive hint refinement; temporal model integration; classroom environment; system integration; teacher correction workflow; security hardening |

| Iteration | Planned work |
|---|---|
| Iteration 4 | Diagnosis, temporal, hint and teacher evaluation experiments; final testing and QA; final refinement and documentation; deployment and release readiness |

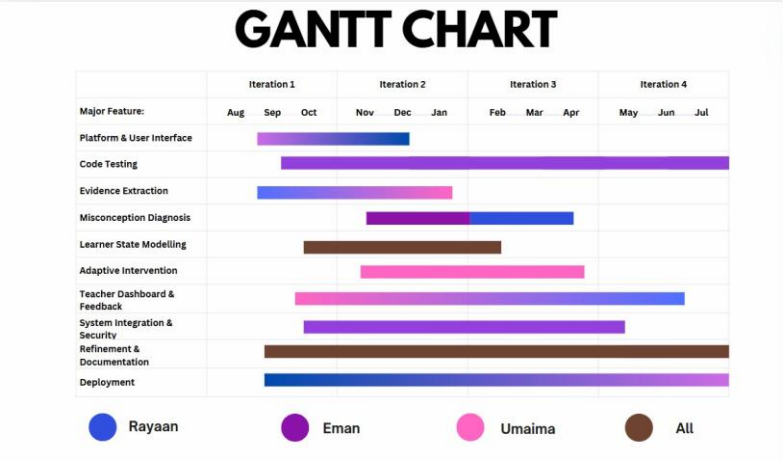



Table 1.5: Iteration 1 Progress

| Element | Status | Evidence |
|---|---|---|
| RBAC | Done | be7897b, fc16065; test_security.py |
| Monaco integration | Done | 5619739 (C++ mode) |
| C++ execution pipeline | Done | 8ef0efb, c30e5aa; 2 s and 256 MB limits |
| Schema normalisation | Done | 21 tables; unique attempt numbers (fb94119) |
| Draft misconception taxonomy | Done | 3 Python and 10 C++ codes (12c3c26) |
| Evidence extraction pipeline | Done (early stage) | 5 evidence families (b88a962, 12c3c26) |
| Submission logging | Done | Source, attempt number, status, test results (b88a962) |
| Testing | Done | 191/191 tests; CI passes; docs/qa |
| Deployment | Configured | https://trace-learn.vercel.app |

Some work from later iterations was completed early: the sandboxed C++ environment, the 40-problem bank, the learner-state model, classrooms and the teacher correction workflow. Table 1.6 gives the blockers of Iteration 1 and their recovery.

Table 1.6: Blockers in Iteration 1

| Blocker | Recovery | Date |
|---|---|---|
| C++ submissions did not run on Windows | Gave the compiled program its full path (fb94119) | 1 Oct 2026 |
| C++ evidence was not scored; correct code was sent to review | Mapped the C++ codes into scoring (12c3c26); added a no-misconception result when all tests pass (fb94119) | 30 Sep 2026 |

| Blocker | Recovery | Date |
|---|---|---|
| Two submissions sent at the same time could get the same attempt number | Added a unique constraint with a retry (fb94119) | 8 Oct 2026 |

## 1.6 Proposal Panel Actions

The proposal decision (14 Sep 2026) was Approved. Table 1.7 gives the one required change and its status.

Table 1.7: Proposal Action Register

| Panel comment | Decision and status | Next action |
|---|---|---|
| "Try to think how you can also use LLM to enhance user experience." | Adapted. Open. An LLM will write the hint text from the evidence, and LLM support will help teachers grade submissions. The LLM will not make the diagnosis. Iteration 1 uses no LLM. | LLM hint prototype in Iteration 2; grading support in Iteration 3 |

# Chapter 2

# Project Requirements

This chapter gives the use cases, the functional and non-functional requirements, and the acceptance criteria of Iteration 1.

## 2.1 Use Cases

TraceLearn has two human actors, the student and the teacher. A use-case model is the most suitable representation. Table 2.1 gives the use cases that Iteration 1 implements, and Figure 2.1 shows them.

Table 2.1: Use Cases for TraceLearn (Iteration 1)

| Title | Actor | Preconditions | Post conditions | Main flow |
|---|---|---|---|---|
| UC1 Register and sign in | Student, Teacher | A teacher has the invite code | A session cookie and a CSRF cookie are set | The user enters email, password (8 or more characters) and name. The system checks the input and creates the account. Sign-in returns the role. |
| UC2 Join a classroom | Student | Signed in; has a join code | The student is a classroom member | The student types the join code. The system adds the student to the class. |
| UC3 Open an assigned C++ problem | Student | The problem is in the bank or assigned to the student | The statement and visible tests are shown | The student selects a problem. The API sends the statement and visible tests only. |
| UC4 Submit C++ code | Student | UC3 is complete | A submission, test results, evidence, two diagnoses and a hint are stored | The student writes code in Monaco and clicks Run / Submit. The server compiles, runs the tests and analyses the result. |
| UC5 View result, diagnosis and hint | Student | UC4 is complete | Hint events are recorded | The result page shows the tests, the possible misconception and hint level 1. The student asks for the next level. |

| Title | Actor | Preconditions | Post conditions | Main flow |
|---|---|---|---|---|
| UC6 Create classroom and assign | Teacher | Signed in as teacher | A classroom and an assignment exist | The teacher creates a class, shares the join code and assigns a bank problem or a new C++ problem. |
| UC7 Review a diagnosis | Teacher | The student is in the teacher's class | A teacher_reviews row exists; the diagnosis is unchanged | The teacher opens the submission, compares the two diagnoses and the evidence, and confirms, rejects or corrects. |
| UC8 View students and misconceptions | Teacher | The class has members | None | The teacher opens the class overview, the misconception list or one student. |
| UC9 Replay synthetic research data | Teacher | ENABLE_DEMO_ENDPOINTS=true | SYN students have new attempts | The teacher resets the demo data and runs a SYN sequence. The research page shows the metrics. |

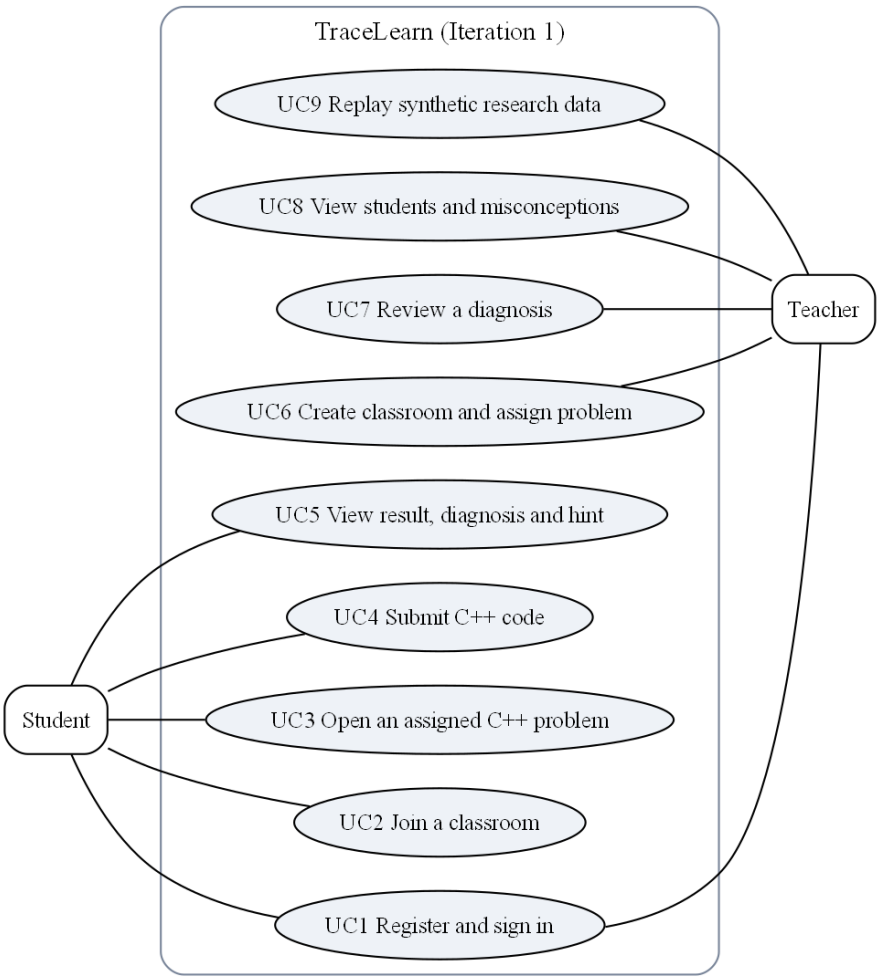



Figure 2.1: Use-Case Diagram

## 2.2 Functional Requirements

This section gives the functional requirements by module. The ID in brackets is the ID used in the evidence worksheet and the acceptance table (Table 2.5).

### 2.2.1 Module 1: Authentication and Role-Based Access

1. The system shall give access that agrees with the role. A student shall not use a teacher-only endpoint, and a teacher shall not make a submission (FR-01).
2. The system shall require a valid invite code to register a teacher account.
3. The system shall return 404 for an item that the user is not permitted to see.

### 2.2.2 Module 2 and 3: Practice Interface and Execution

1. The system shall send the statement and the visible tests of an assigned C++ problem, and never the hidden test input or output (FR-02).
2. The system shall store each submission with the source, the problem ID and an attempt number, and send back an ID (FR-04).

3. The system shall compile and run C++ with limits: correct code gives `success`, code that does not compile gives `syntax_error`, wrong output gives `wrong_answer`, and an infinite loop gives `timeout` at the test limit (FR-05).
4. The system shall run the visible, hidden and diagnostic tests, and each stored result shall link to a test case with a type (FR-06).

### 2.2.3 Module 4 and 5: Evidence, Diagnosis and Learner State

1. The system shall extract evidence from each submission from at least three evidence families (FR-08).
2. The system shall make a current-only and a history-aware diagnosis with a confidence and an uncertainty level for each submission.
3. The system shall change the learner state and record the reason. One failure shall never make a PERSISTENT state; support on two or more different problems can make it (FR-11).

### 2.2.4 Module 6 and 7: Hints, Review, Classrooms and CI

1. The system shall give the first hint at level 1 and the next level on request, and shall record each hint.
2. The system shall store a teacher review without changing the original diagnosis.
3. CI shall check each change; a pull request with a failed test shall not pass its checks (FR-10).

## 2.3 Non-Functional Requirements

This section gives the non-functional requirements. Each requirement is measurable and has a test or a measured result.

### 2.3.1 Security

Table 2.2: Security Requirements

| ID | Requirement | Description and measurement |
|---|---|---|
| NFR-01 | Safe failure | Compile, runtime, timeout and output-limit cases give a status. The API continues to operate and does not show internal data (server paths, stack traces). |
| NFR-02 | Isolation | Student code cannot read host files, use the network or read environment variables (Linux with bubblewrap). On Windows there is no isolation; Windows is for development only. |
| NFR-03 | No future information | A diagnosis uses only events that occurred before the submission (test_no_future_history_leakage). |
| SEC-4 | Input limits | Request body at most 256 KB; source code at most 64 KB; rate limits on sign-in, registration and execution. |

### 2.3.2 Performance

Table 2.3: Performance Requirements

| ID | Requirement | Description and measurement |
|---|---|---|
| PER-1 | Execution limits | Each bank test runs with a 2,000 ms wall-clock limit and a 256 MB memory limit. Measured: an infinite loop stops after 2,003 ms; a 512 MB request is stopped (MemoryLimitExceeded). |
| PER-2 | Reference run time | Reference solutions run in a median of 4.63 ms and a maximum of 10.34 ms (verification report, 8-problem bank). |
| PER-3 | Concurrency | At most 4 submissions run at the same time (MAX_CONCURRENT_EXECUTIONS). |

## 2.4 Acceptance Criteria and Traceability

Table 2.4: Requirements and Acceptance Evidence

| ID | Acceptance criterion | Status | Evidence |
|---|---|---|---|
| FR-01 | A student cannot use a teacher-only endpoint. A teacher cannot make a submission. | Met | test_research_endpoints_teacher_only; test_teachers_cannot_create_submissions; test_every_endpoint_requires_auth |
| FR-02 | The API sends the statement and visible tests, never hidden test input or output. | Met | test_custom_problem_visibility_and_assignment; test_students_never_see_hidden_expected_outputs |
| FR-04 | POST /api/submissions stores the source, problem ID and attempt number and sends back an ID. | Met | test_concurrent_submissions_get_distinct_attempt_numbers; docs/qa/api_qa.py |
| FR-05 | Correct → success; no compile → syntax_error; wrong output → wrong_answer; infinite loop → timeout at 2,000 ms. | Met | test_cpp_correct_solution_passes; test_cpp_program_runs_without_bubblewrap; docs/PROBLEM_VERIFICATION_REPORT.md; docs/qa/bank_e2e_results.json |
| FR-06 | Each stored result links to a test case with a type. No diagnostic test is public. | Met | TestResult.test_case_id foreign key; test_no_diagnostic_test_is_public |
| FR-08 | At least 3 evidence families give the same output each time on the fixtures. | Partly met | 5 families give evidence (docs/qa/bank_e2e_results.json). No test runs the same input twice and compares the output. |
| FR-10 | A pull request with a failed test does not pass its required checks. | Met | .github/workflows/ci.yml fails when a test fails. The main branch has no branch protection, so no check is required. |
| FR-11 | One failure never makes PERSISTENT; support | Met | test_single_failure_not_automatically_persistent; test_cross_problem_recurrence_can_create_persistent_state |

| ID | Acceptance criterion | Status | Evidence |
|---|---|---|---|
| | on 2 or more problems can; each change writes a history row. | | |
| NFR-01 | Failure cases give a status; the API continues; no internal data is shown. | Met | docs/qa/api_qa.py; test_validation_errors_do_not_echo_input |
| NFR-02 | No access to host files, network or environment variables. | Met | test_sandbox_blocks_files_network_and_env; docs/qa/sandbox_probe.py |

Table 2.5: Requirement Traceability

| Requirement | Design element | Implementation | Demo step |
|---|---|---|---|
| FR-01 | Activity A1; architecture: Student and Teacher UI | api/auth.py; auth/access.py; auth/dependencies.py | 1 |
| FR-02 | Activity A2; ERD: problems, test_cases, assignments | api/problems.py; api/classrooms.py; pages/student/Problem.tsx | 2 |
| FR-04 | Activity A3–A4; ERD: submissions | api/submissions.py; services/submission_service.py | 3 |
| FR-05 | Activity A5; architecture: Sandbox + test runner | execution/cpp_runner.py; execution/sandbox.py | 3, 6 |
| FR-06 | Activity A6; ERD: test_results | execution/test_runner.py; data/cpp_diagnostics | 3 |
| FR-08 | Activity A7; architecture: Evidence extractor + fusion | analysis/pipeline.py; analysis/cpp_static.py; diagnosis/cpp_mapping.py | 4 |
| FR-10 | CI/CD pipeline (docs/CI_CD.md) | .github/workflows/ci.yml | 6 |
| FR-11 | Activity A9; state diagram; ERD: learner_states | learner_model/updater.py; learner_model/history.py | 5 |
| NFR-01, NFR-02 | Activity A5 failure branch; sandbox boundary | execution/sandbox.py; execution/cpp_runner.py | 6 |

# Chapter 3

# System Overview

This chapter gives the architecture, the component interfaces, the data design, the process and behaviour design of Iteration 1, and the changes since the proposal.

## 3.1 Architectural Design

TraceLearn has three layers: the user layer, the application services and the diagnostic intelligence. Figure 3.1 shows the layers and how a submission becomes evidence, a diagnosis and a hint.

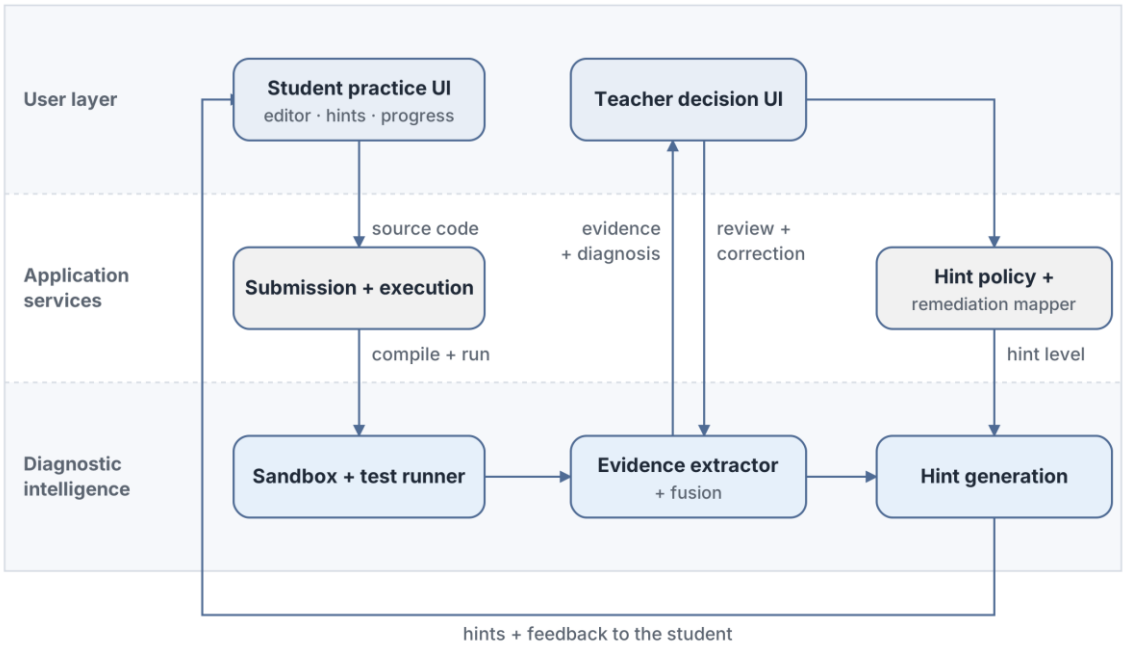



Figure 3.1: System Architecture of TraceLearn (v2, October 2026)

### 3.1.1 User Layer

- **Student practice UI** (`frontend/src/pages/student`): the Monaco editor, the result page with hints and the progress page.
- **Teacher decision UI** (`frontend/src/pages/teacher`): classrooms, assignments, the submission page with both diagnoses and the review form.
- React, Vite and TypeScript. The frontend selects pages from the role that the server sends; the backend also checks each call.

### 3.1.2 Application Services

- **Submission and execution** (`api/submissions.py`, `services/submission_service.py`): stores the submission and calls execute, analyze and diagnose in order.
- **Hint policy** (`hints/hint_policy.py`, `hints/templates.py`): selects the hint target and level.
- **Authentication, classrooms and security** (`api/auth.py`, `auth/`, `api/classrooms.py`, `security/`).

### 3.1.3 Diagnostic Intelligence

- **Sandbox and test runner** (`execution/sandbox.py`, `cpp_runner.py`, `test_runner.py`).
- **Evidence extractor and fusion** (`analysis/`, `diagnosis/`): five analyzers make evidence items; `diagnosis/confidence.py` fuses them into two diagnoses.
- **Learner model** (`learner_model/updater.py`, `history.py`): updates the state and reads history only up to the submission time.

### 3.1.4 Component Interfaces

The frontend calls the FastAPI backend over HTTP with a session cookie and a CSRF token. Table 3.1 gives the main endpoints of Iteration 1.

Table 3.1: Main API Endpoints

| Method and path | Who can use it | Function |
|---|---|---|
| POST /api/auth/register, /login | Public (rate limit) | Create an account; sign in |
| GET /api/problems/{id} | Signed-in user | Problem with visible tests only for students |
| POST /api/submissions | Student | Store a submission for the signed-in student |
| POST /api/submissions/{id}/run-and-analyze | The student who submitted | Execute, analyze and diagnose |
| GET /api/submissions/{id}/evidence, /diagnoses, /test-results | The student or the student's teacher | Read the results |
| POST /api/submissions/{id}/next-hint | The student who submitted | Reveal the next hint level |
| POST /api/diagnoses/{id}/teacher-review | The student's teacher | Store a review |
| POST /api/classrooms, /classrooms/join, /classrooms/{id}/assignments | Teacher; student (join) | Classrooms and assignments |
| GET /api/evaluation/comparison | Teacher | Experiment metrics |

### 3.1.5 Design Decisions

Table 3.2: Architecture and Interface Decisions

| Decision | Alternatives | Chosen option | Trade-off |
|---|---|---|---|
| How to run student C++ | Trusted subprocess; Docker; browser execution | g++ in bubblewrap with CPU, memory and file limits and a wall-clock stop; production refuses to run without the sandbox | Isolation without a container daemon; Windows development computers use only the limits |
| How to find a misconception | Rule-based; XGBoost; LLM | Rule-based fusion in Iteration 1; XGBoost in Iteration 2 | Clear and repeatable; XGBoost needs labelled data; LLM not used for the diagnosis because it can give unsupported results |
| Use of student history | One model with history features | Two engines (current-only and history-aware) on each submission | Direct comparison for the research question; two code paths to keep |
| How to make hints | LLM-written hints; one fixed hint | Template hints at 3 levels; problem-specific hints replace them | The hint never shows the solution; new misconceptions need new templates |
| How to store teacher corrections | Overwrite the diagnosis | A teacher_reviews row; the diagnosis never changes | Full audit trail; reads join two tables |

## 3.2 Data Design

The database has 21 tables (`backend/app/db/models.py`). SQLite stores the data; the models also operate with PostgreSQL. Figure 3.2 shows the entity relationships and Figure 3.3 shows the columns.

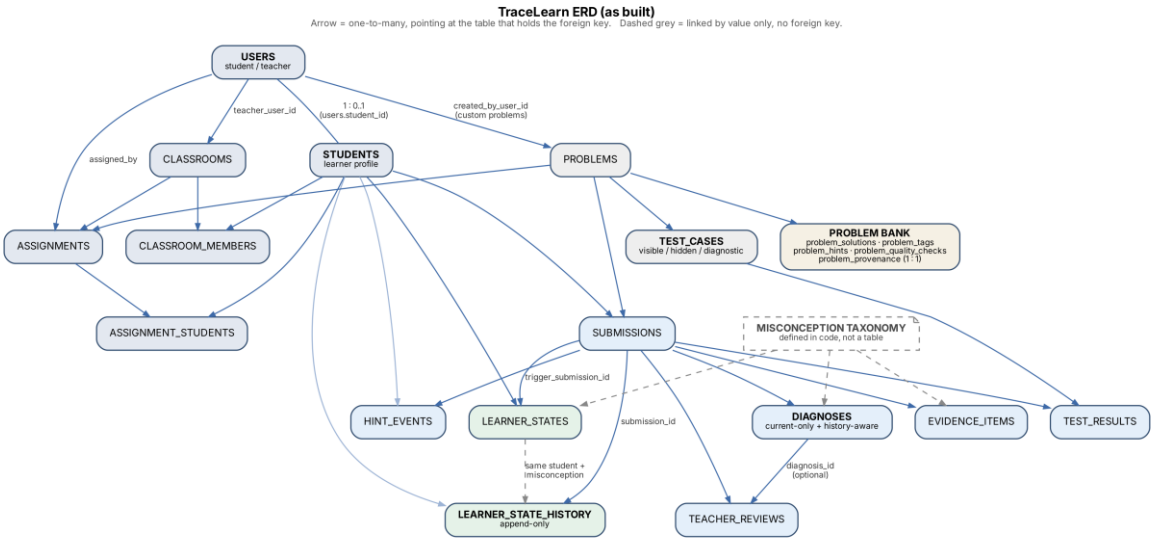



Figure 3.2: Entity-Relationship Diagram of TraceLearn (as built)

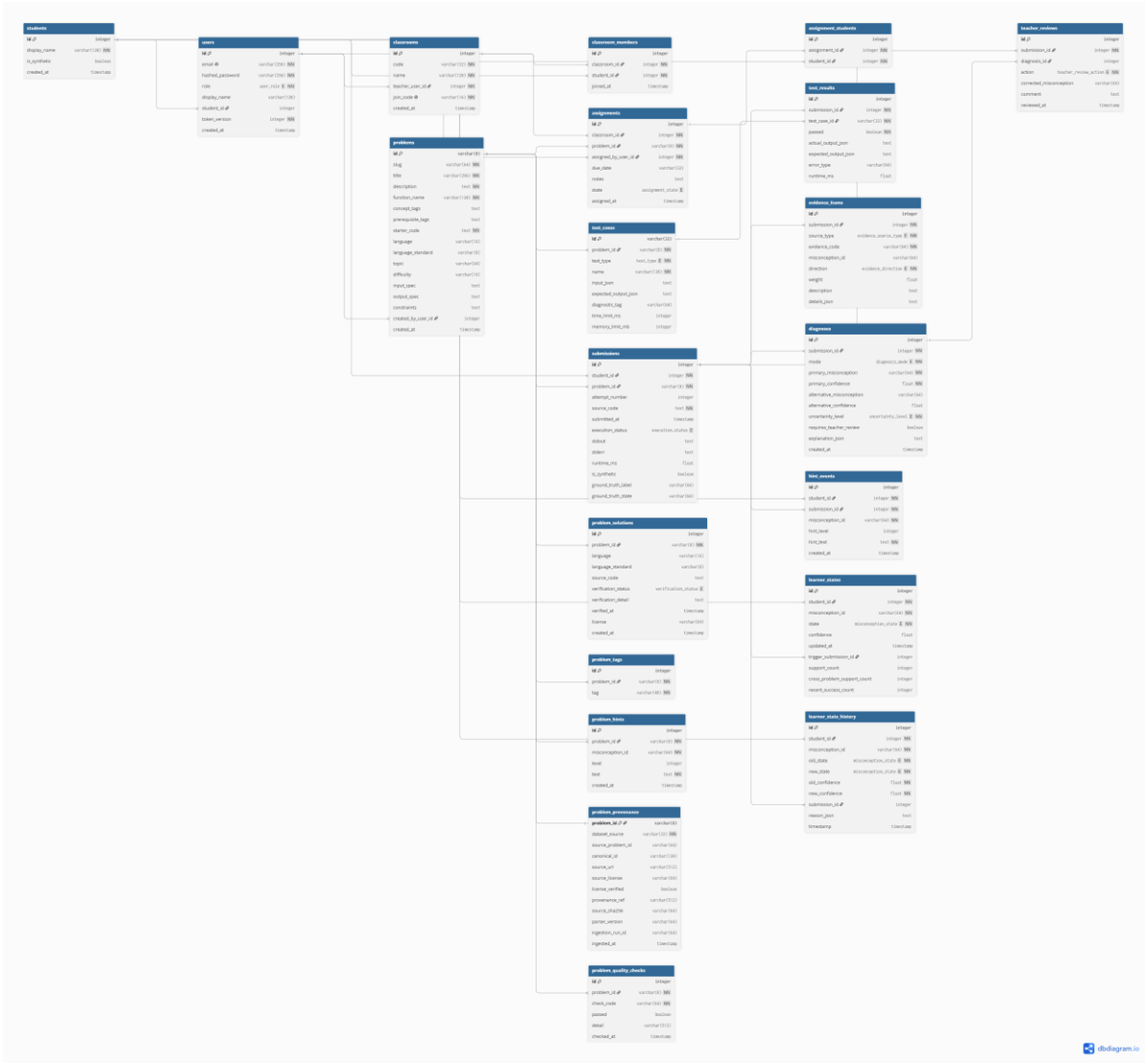


Figure 3.3: Database Schema of TraceLearn

Link: https://dbdiagram.io/d/6aca1cd250a8c6fc361b5c83

Table 3.3: Key Entities and Their Relationships

| Group | Tables | Purpose |
|---|---|---|
| Identity and classes | users, students, classrooms, classroom_members, assignments, assignment_students | Login identity is separate from learner data; RBAC uses classroom membership and assignments |
| Problems | problems, test_cases, problem_solutions, problem_tags, problem_hints, problem_provenance, problem_quality_checks | Problem bank with visible, hidden and diagnostic tests, reference solutions, provenance and quality checks |
| Attempts and evidence | submissions, test_results, evidence_items, diagnoses, hint_events | One row per attempt (unique student, problem, attempt number); results and evidence of each attempt |
| Learner model | learner_states, learner_state_history, teacher_reviews | State for each student and misconception; append-only history with reasons; reviews |

The misconception taxonomy is defined in code (`diagnosis/taxonomy.py`, `diagnosis/cpp_mapping.py`), not in a table. It has 3 Python codes (M01 loop boundary, M02 index and value, M03 missing return) and 10 C++ codes. The bank codes CPP-M01 to CPP-M08 map to these codes.

## 3.3 Process Design

Figure 3.4 shows the steps of one submission across the student, the API, the sandbox and the analysis. Steps A1 to A12 agree with `POST /api/submissions` followed by `/run-and-analyze` (SubmissionService.execute → analyze → diagnose).

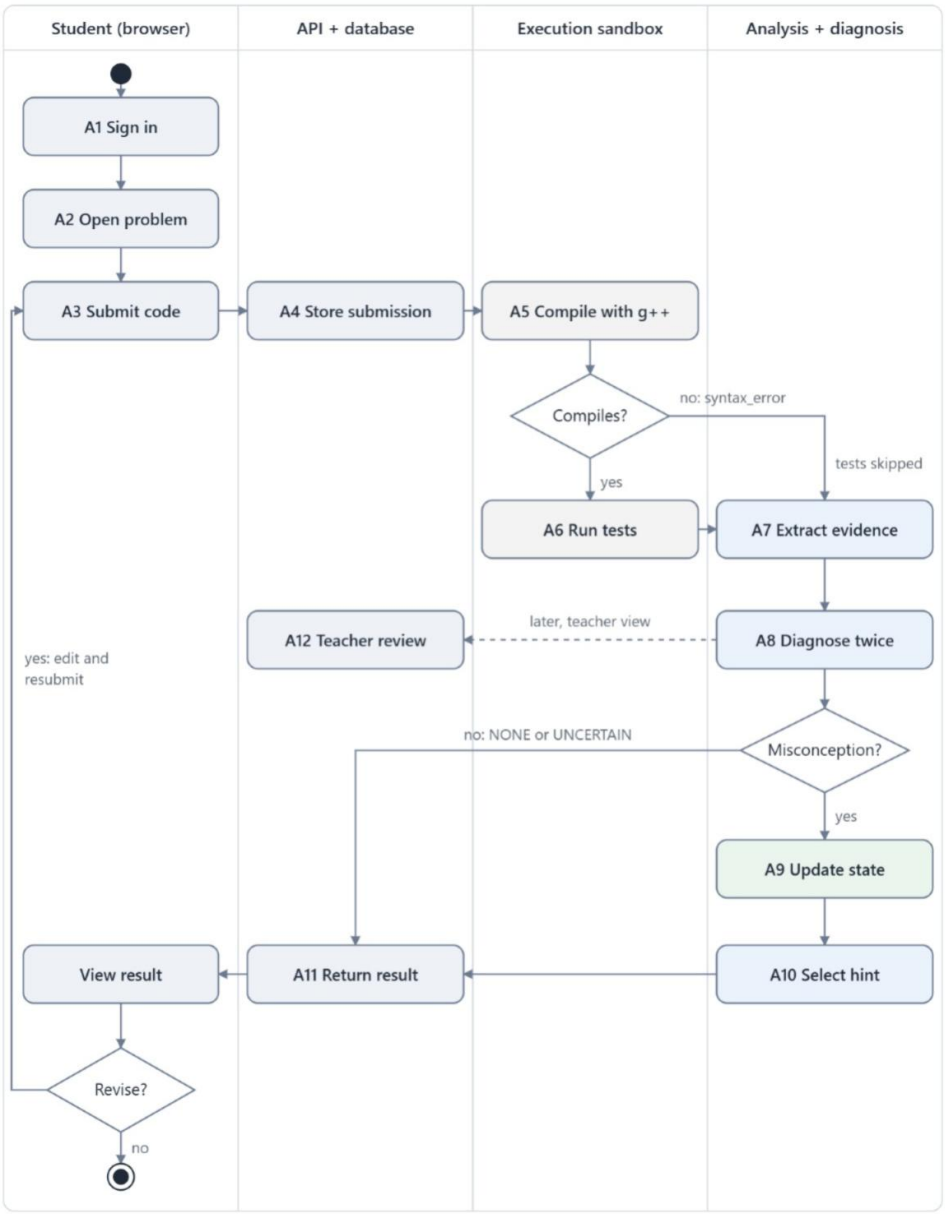



Figure 3.4: Activity Diagram of One Submission

If the code does not compile, no tests run and the evidence step receives a syntax error. If the diagnosis is NO_MISCONCEPTION or UNCERTAIN, the learner state does not change.

## 3.4 Behaviour Design

Figure 3.5 shows how the misconception state of a student changes, as implemented in `learner_model/updater.py` (`_compute_state`). The rules run in this order: teacher review; RECURRED; IMPROVING; RESOLVED; PERSISTENT; otherwise NEW.

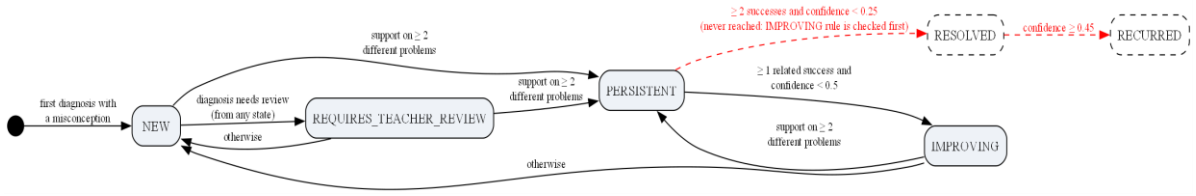



Figure 3.5: State-Transition Diagram of a Learner Misconception State

## 3.5 Design Evolution

Table 3.4: Design Artefacts and Changes since the Proposal

| Artefact | Change since proposal | Rationale |
|---|---|---|
| Architecture | teacher review loop added | The stakeholders selected C++. Blueprint section 16.1 requires isolation before students use the system. |
| Data / ERD | A separate students table; classroom, assignment and problem-bank tables added | Login identity is separate from learner data; RBAC needs assignments; the bank keeps provenance and quality checks. |
| Process | Compile-error branch (no tests run) and no-misconception branch (no state change) added | The diagram agrees with the code for the live demonstration. |
| Behaviour | RESOLVED and RECURRED shown as not reachable | The diagram shows the code as it is. |
| C++ diagnostic bank | New after the proposal: 40 problems for CPP-M01 to CPP-M08 (305 tests, 32 variants, 120 hints) | The diagnostic tests separate the misconceptions and are the main input to evidence extraction. |

# Chapter 4

# Implementation and Testing

This chapter documents the completed core module: the setup, the main files and functions, the algorithms, the technical decisions, the tests and the disclosures.

## 4.1 Setup and Execution

Requirements: Python 3.11 or later, Node.js 18 or later, g++ with C++17, and bubblewrap on Linux for isolation. Windows can run the system for development, without isolation. These commands run in the repository folder:

1. `python -m pip install -r requirements.txt`
2. `cd frontend && npm install`
3. Copy `.env.example` to `.env`; set `TEACHER_SIGNUP_CODE` and `SECRET_KEY` (32 or more characters in production).
4. `python scripts/seed_database.py` and `python scripts/seed_cpp_diagnostics.py` (46 problems in total).
5. `uvicorn backend.app.main:app --port 8000`, then `cd frontend && npm run dev`, and open http://localhost:5173.
6. Tests: `python -m pytest backend/tests -q` (set `SANDBOX_MODE=bwrap` on Linux).

   Or

   https://trace-learn.vercel.app/

## 4.2 Main Files and Functions

Table 4.1: Main Files of the Core Module

| File | Main classes or functions | Function |
|---|---|---|
| execution/cpp_runner.py | compile_cpp, CppCompileSession.run | Compile once with g++; run each test with stdin; map the result to a status |
| execution/sandbox.py | run_in_sandbox, check_cpp_source, bwrap_path | Start the process in bubblewrap with limits; reject unsafe #include |
| execution/test_runner.py | run_submission_tests | Run all tests, compare tokens, store test_results, set the overall status |
| analysis/pipeline.py | AnalysisPipeline.analyze_submission | Run the five analyzers and store evidence_items |

| File | Main classes or functions | Function |
|---|---|---|
| analysis/cpp_static.py | CppStaticAnalyzer | Regular-expression patterns and g++ warnings for C++ |
| diagnosis/confidence.py | apply_evidence_weights, fuse_diagnosis | Weight evidence, score misconceptions, set confidence and review |
| diagnosis/cpp_mapping.py | code_for_diagnostic_tag | Map bank tags and codes to backend codes |
| learner_model/updater.py | LearnerStateUpdater._compute_state | Update the state and write a history row with the reason |
| hints/hint_policy.py | hint_levels, HintPolicy.select_hint | Select the hint source and the first level |
| services/submission_service.py | create_submission, run_and_analyze | Allocate the attempt number; run the full pipeline |

## 4.3 Algorithm Design

This section gives the main algorithms of the core module. Table 4.2 lists them.

Table 4.2: Summary of Algorithms Implemented in Iteration 1

| No. | Algorithm | Reference |
|---|---|---|
| 1 | Sandboxed C++ Compilation and Test Execution | Algorithm 1 |
| 2 | Submission Analysis Pipeline | Algorithm 2 |
| 3 | Evidence Fusion and Diagnosis | Algorithm 3 |
| 4 | Learner-State Update | Algorithm 4 |
| 5 | Hint Selection | Algorithm 5 |

### 4.3.1 Sandboxed C++ Compilation and Test Execution

**Algorithm 1** Sandboxed C++ Compilation and Test Execution

**Require:** source code *s*, test cases *T* of the problem
**Ensure:** overall status, one result per test

```
if size(s) > 64 KB or s includes a non-standard header then
    return syntax_error
end if
binary ← g++ -std=c++17 -O2 -Wall s  {in bubblewrap, 2 GB compiler memory}
if compile fails then return syntax_error with cleaned compiler output
status ← success
for each test t in T do
    r ← run(binary, stdin = t.input, time = t.limit (2,000 ms), memory = 256 MB)
    passed ← r.ok and tokens(r.stdout) = tokens(t.expected)
    store test_result(t, passed)
    if r is timeout then status ← timeout; stop
    if r is a crash then status ← runtime_error
```

13: end for
14: if status = success and a test failed then status ← wrong_answer
15: return status

### 4.3.2 Submission Analysis Pipeline

**Algorithm 2** Submission Analysis Pipeline

**Require:** student *u* (from the session), problem *p*, source *s*
**Ensure:** stored submission, evidence, two diagnoses, learner state, hint

1: n ← max(attempt_number of u on p) + 1 {unique constraint; retry on conflict}
2: sub ← store(u, p, s, n)
3: execute(sub) {Algorithm 1}
4: E ← test, static, runtime, diff and history evidence for sub
5: {history uses only data before sub.submitted_at}
6: d_cur ← fuse(E without sequence and history) {Algorithm 3}
7: d_hist ← fuse(E)
8: update learner state from d_hist {Algorithm 4}
9: select hint from d_hist {Algorithm 5}
10: return sub, E, d_cur, d_hist

### 4.3.3 Evidence Fusion and Diagnosis

**Algorithm 3** Evidence Fusion and Diagnosis

**Require:** evidence records *E*, weights from evidence_weights.yaml
**Ensure:** primary misconception, confidence, uncertainty, review flag

1: for each e in E do e.weight ← weight table[e.code] {e.g. static 0.30, diagnostic 0.35}
2: for each misconception m do score[m] ← Σ support − 0.5 · Σ |conflict|
3: if E has a syntax error then return NO_MISCONCEPTION, 0.1, review
4: if all tests passed then return NO_MISCONCEPTION, 0.80, no review
5: m ← *argmax score; c ← clamp(score[m*], 0, 1)
6: if c < 0.35 then return UNCERTAIN, review
7: if two scores are within 0.10 or conflict evidence exists then review ← true
8: uncertainty ← high if review, medium if c < 0.55, else low
9: return m*, c, uncertainty, review

### 4.3.4 Learner-State Update

**Algorithm 4** Learner-State Update

**Require:** history-aware diagnosis *d*, current state *S* for (student, *m*)
**Ensure:** new state and a history row with the reason

1: if d is NO_MISCONCEPTION or UNCERTAIN then return {no change}
2: if d needs review then new ← REQUIRES_TEACHER_REVIEW
3: else if S = RESOLVED and d.confidence ≥ 0.45 then new ← RECURRED
4: else if S = PERSISTENT and successes ≥ 1 and d.confidence < 0.5 then new ← IMPROVING
5: else if S = PERSISTENT and successes ≥ 2 and d.confidence < 0.25 then new ← RESOLVED {not reachable}
6: else if support on ≥ 2 different problems then new ← PERSISTENT
7: else new ← NEW
8: store state and append history(old, new, reason)

### 4.3.5 Hint Selection

**Algorithm 5** Hint Selection

**Require:** history-aware diagnosis *d*, submission *sub*
**Ensure:** a recorded hint event at level 1

```
if d is NO_MISCONCEPTION and no review then return none
if d is UNCERTAIN, NO_MISCONCEPTION or needs review then levels ← {1: generic hint}
else if problem hints exist for d (bank or backend code) then levels ← problem hints
else levels ← template hints for d
record hint event at level min(levels)
    {next-hint reveals the next level on request}
```

## 4.4 Technical Decisions and Limitations

- The evidence weights are fixed values set by the team (version poc-v1). They are not yet validated with real data.
- The C++ static analysis uses regular expressions and g++ warnings, not a full AST.
- Isolation operates only on Linux with bubblewrap. On Windows the code runs without isolation and without memory or output limits.
- The history-aware mode did not change any prediction in the synthetic experiment (history_changed_predictions = 0).
- RESOLVED and RECURRED cannot be reached (Section 3.4).
- When diagnostic tests disagree, the attempt always goes to teacher review. This affects 10 of 43 bank variants. It is an open design decision (labelled case SYN005).
- All data is synthetic. There are no real students yet.

## 4.5 External APIs, Tools and Dependencies

The system uses no external API at runtime. Table 4.3 gives the tools and libraries.

Table 4.3: External Tools and Dependencies

| Tool or library | Purpose | Validation | Risk and fallback |
|---|---|---|---|
| g++ (C++17) | Compile student code | 77/77 reference tests; 40/40 bank solutions | The version differs between computers; record the version and use the verified computer |
| bubblewrap | Process isolation | test_sandbox_blocks_files_network_and_env; sandbox_probe.py | Linux only; production refuses to run without it |
| FastAPI, SQLAlchemy, SQLite, Pydantic | API and storage | 191 backend tests | SQLite allows one write at a time; the models also run on PostgreSQL |
| python-jose, | Tokens and | test_forged_and_tampered_tokens_rejected | python-jose gets few |

| Tool or library | Purpose | Validation | Risk and fallback |
|---|---|---|---|
| passlib (bcrypt) | password hashes | | updates; replace with PyJWT |
| React 19, Vite 8, Monaco 0.52 | Student and teacher UI | npm run build in CI | Fast-moving versions; lockfile and npm ci |
| Project CodeNet, PROGpedia, CodeStream | C/C++ code for ingestion | File-level audit; verify_normalized.py | No misconception labels; the team makes its own labels |

## 4.6 Testing Details

The backend is tested with pytest and the frontend with oxlint and the TypeScript build. CI runs the backend tests on Python 3.11 and 3.12 with g++ and bubblewrap. The team also ran an end-to-end QA against a live backend on 10 October 2026 (docs/qa).

### 4.6.1 Unit and Contract Testing

Table 4.4: Backend Test Suites

| Test file | Tests | Area |
|---|---|---|
| test_security.py | 30 functions (more cases with parameters) | Authentication, access between users, CSRF, input limits, sandbox |
| test_cpp_diagnostics_bank.py, test_cpp_problem_bank.py | 55 functions | Shape, checksums and verification of the C++ banks |
| test_cpp_end_to_end.py | 10 | C++ variant → evidence → diagnosis → learner state → hint |
| test_iteration1_regressions.py | 8 functions (13 cases) | One test for each QA defect |
| test_diagnosis.py, test_history_model.py, test_state_transitions.py, test_mandatory_contracts.py | 15 | Diagnosis modes, no future leakage, learner states |
| test_execution.py, test_static_analysis.py, test_milestone1.py, test_problem_bank.py | 20 | Execution, static patterns, seed data, bank API |
| **Total collected by pytest** | **191** | 189 pass and 2 skip on Windows (bubblewrap only) |

Table 4.6 gives selected test cases in the same format as the unit tests of the Kairo report.

Table 4.5: Selected Test Cases

| ID | Objective | Steps and data | Expected result | Actual result | Pass/Fail |
|---|---|---|---|---|---|
| TC001 | A student cannot use a | Student calls GET /api/evaluation/comparison and POST | 403 | 403 | Pass |

| ID | Objective | Steps and data | Expected result | Actual result | Pass/Fail |
|---|---|---|---|---|---|
| | teacher endpoint | /api/demo/reset | | | |
| TC002 | A teacher cannot submit code | Teacher calls POST /api/submissions | 403 | 403 | Pass |
| TC003 | Hidden tests stay hidden | Student opens CP01; then submits wrong code | No input or output for hidden and diagnostic tests | 3 visible tests with data; 6 hidden or diagnostic tests with no data | Pass |
| TC004 | Correct C++ passes | Reference solution of CP01 | success; 9/9 tests; NO_MISCONCEPTION | success; 9/9; NO_MISCONCEPTION, no review | Pass |
| TC005 | Compile error | `std::cout << x;` with undeclared x | syntax_error; no server paths in the message | syntax_error; message has no /tmp or /home path | Pass |
| TC006 | Infinite loop | `while(true){++c;}` | timeout; API continues | timeout; /health answers | Pass |
| TC007 | Sandbox isolation | C++ probe opens /etc/passwd, .env, the database; connects to 8.8.8.8 | All blocked | All blocked; process is PID 2 in its own namespace | Pass (Linux) |
| TC008 | Unique attempt numbers | 8 parallel POST /api/submissions | Attempts 1 to 8 | [1..8] (before the fix: [1,2,2,2,2,4,7,7]) | Pass |
| TC009 | Hard negative | CP01 hard negative with `i <= limit` | All tests pass; no misconception | 9/9; NO_MISCONCEPTION | Pass |
| TC010 | One failure is not PERSISTENT | One off-by-one attempt | State is not PERSISTENT | NEW / REQUIRES_TEACHER_REVIEW | Pass |

### 4.6.2 Integration Testing

The QA script `docs/qa/bank_e2e.py` submitted every reference solution, misconception variant and hard negative of the C++ bank through the live API. The script `docs/qa/api_qa.py` ran 77 checks on RBAC, CSRF, the C++ pipeline, evidence and logging. Table 4.7 gives the results before and after the QA fixes.

Table 4.6: Testing Statistics Overview

| Check | Before (12c3c26) | After (fb94119) |
|---|---|---|
| pytest, Linux with bubblewrap | 166/168 | 191/191 |
| pytest, Windows | 159/168 (C++ could not | 189 pass, 2 skip |

| Check | Before (12c3c26) | After (fb94119) |
|---|---|---|
| | run) | |
| API checks (api_qa.py) | 75/77 | 77/77 |
| Correct solutions → NO_MISCONCEPTION | 18/40 (15 sent to review) | 40/40 (0 sent to review) |
| Hard negatives → NO_MISCONCEPTION | 0/4 | 4/4 |
| Misconception variants with the correct label | 38/43 | 40/43 |
| Synthetic experiment accuracy (both modes) | 0.47 | 0.87 |
| 8 concurrent submissions | [1,2,2,2,2,4,7,7] | [1..8] |

### 4.6.3 Live Demonstration Plan

Table 4.7: Live Demonstration Steps

| Step | Action | Expected result | Presenter |
|---|---|---|---|
| 1 | Sign in as a student | The student dashboard opens; teacher pages are blocked | Eman |
| 2 | Open an assigned C++ problem | Statement and visible tests only | Umaima |
| 3 | Submit incorrect C++ (index equal to the vector size) | The code compiles; some tests fail; the status is shown | Umaima |
| 4 | Open the evidence and diagnosis | Evidence items, two diagnoses with confidence, and a hint | Rayaan |
| 5 | Submit corrected code | Attempt 2 is stored; the tests pass | Rayaan |
| 6 | Submit an infinite loop; show CI | timeout at the limit; the API continues; the CI run passes | Umaima |

### 4.6.4 Failure Case Discussion

A student submits a recursive factorial that never changes its argument (CP07 variants). Code that does not stop can block the server for all students, and it is evidence for CPP-M07. The verifier stops the process at the 2,000 ms limit (return code -9) and 5 of 9 tests fail. In the application the status is `timeout`: 2 tests pass, 1 times out and the remaining tests do not run. The diagnosis is CPP_MISSING_BASE_CASE. The next submissions run with no change. A timeout is one signal that supports the misconception; it is not a diagnosis by itself.

## 4.7 Risk, Security and Privacy

- **Security controls**: JWT in an httpOnly cookie; CSRF token on each write; bcrypt hashes; sign-out cancels tokens (token_version); rate limits; security headers; CORS allow-list; production startup checks (SECRET_KEY of 32 or more characters, demo endpoints off).

- **Sensitive data**: the database stores the student email, display name and submitted code. Research data uses separate synthetic students. The team must write the consent, retention and deletion policy before tests with real students.
- **Availability**: if bubblewrap is missing, production does not run code. If the frontend host stops, the frontend can run on a local computer.
- **Process risk**: the main branch has no branch protection, so CI checks are not required before a merge.

## 4.8 GenAI Use Disclosure

The team used GenAI tools for parts of the work. Table 4.9 gives the highest assistance level for each artefact (L1 supportive, L2 assisted, L3 core assist, L4 substituted), from the evidence worksheet.

Table 4.8: GenAI Artefact Register

| Artefact | Tool and level | Student contribution and verification | Owner |
|---|---|---|---|
| Initial POC | Cursor; L2 | Design changed to HCI principles; functions defined step by step | Rayaan |
| Frontend and backend integration | Claude; L3 | Replaced hard-coded data with the real API, sign-in and role routing; npm run build | Umaima |
| Dataset audit and ingestion | OpenCode; L2 | Explained the schema; normalised the data; verify_normalized.py | Rayaan |
| C++ diagnostic bank content | OpenCode; L4 for the bank data, L2 for the engineering | Independent models check every expected output; checksums; 4/4 hard negatives | Rayaan |
| Sandbox and security test set | Claude; L2 | Manual and iterative QA; failing cases fixed | Eman |
| Iteration 1 QA fixes (fb94119) | Claude; L2 | Rejected one change that broke SYN005; full test suite; bank E2E; experiment before and after | Eman |
| CI/CD workflows | Claude; L3 | actionlint; GitHub run 38052478077 passed | Umaima |
| README and documents (ASD-STE100) | ChatGPT; L1 | Numbers taken from the team's runs | All |

## 4.9 Research and Citation Verification

The team checked each research claim against the original source. The main corrections: the McMining title is "Automated Discovery" and its benchmark has 1,675 samples, of which 1,063 have a misconception; the samples are synthetic and Python only. The Project CodeNet

data licence is CDLA-Permissive-2.0, not Apache-2.0 (the code licence). PROGpedia labels are verdicts, not misconceptions. CodeStream also contains Java, and C++ is its smallest part.

# Chapter 5

# Conclusions and Future Work

## 5.1 Conclusion

Iteration 1 delivered the planned C++ vertical slice. An authenticated student submits C++ in the Monaco editor. The server compiles and runs it in a bubblewrap sandbox, runs visible, hidden and diagnostic tests, extracts evidence, makes current-only and history-aware diagnoses, updates the learner state and selects a hint. A teacher can review each diagnosis. All 191 backend tests pass on Linux and CI passes. The QA on 10 October 2026 found and fixed defects in Windows execution, scoring, statuses, hints and attempt numbers. Three items remain open: the history-aware mode does not yet change any prediction, the RESOLVED and RECURRED states cannot be reached, and the panel action on the LLM is not started.

## 5.2 Future Work

- **LLM hint prototype** (panel action): an LLM rewrites the template hints from the evidence without showing the solution. The LLM does not make the diagnosis.
- **History-aware diagnosis**: give history evidence its own weight and rerun the synthetic experiment.
- **Learner states**: reorder the state rules so that RESOLVED and RECURRED can be reached, with a test for each change.
- **XGBoost classifier** when labelled data is available, and student testing with 5 to 10 students before the FYP-II pilot.
- **Process**: protect the main branch so that CI must pass before a merge; add C++ tests for the output limit and the timeout.

# References

[1] E. Al-Hossami and R. Bunescu, "McMining: Automated Discovery of Misconceptions in Student Code," in Proc. 19th Conf. of the European Chapter of the Association for Computational Linguistics (EACL 2026), Vol. 2: Short Papers, pp. 160–178, 2026. https://aclanthology.org/2026.eacl-short.10/

[2] McMiner code and data, MIT licence. https://github.com/taisazero/mcminer

[3] IBM, Project CodeNet. https://github.com/IBM/Project_CodeNet

[4] J. C. Paiva, J. P. Leal and Á. Figueira, "PROGpedia: Collection of source-code submitted to introductory programming assignments," Data in Brief, 2023, doi:10.1016/j.dib.2023.108887. Data: https://zenodo.org/records/7449056

[5] N. J. Lina et al., "CodeStream: A dataset of iterative programming submissions with sequential verdict traces and attempt histories," Data in Brief, vol. 66, 2026, doi:10.1016/j.dib.2026.112843. Data: https://data.mendeley.com/datasets/n77t7z9zcr/1

[6] TraceLearn repository. https://github.com/Rayaan-Raza/TraceLearn